



Chapter 5

Designing SQL Server Indexes

Compiled by: Mohammed A.

Chapter Outline

- Introduction
- Index Structure
- Designing Indexes
 - ✓ Clustered Indexes
 - ✓ Nonclustered Indexes
 - ✓ XML Indexes
 - ✓ Spatial Indexes
- Index Management and Maintenance

Introduction

- SQL Server accesses data using an **index** or a **table scan** .
- **Index** is a data structure that can be used to search data more quickly and efficiently.
 - ✓ Speed up searches and optimizes performance.
- When using an index, SQL Server will use the index pages to find the required rows.
- An index is useful only if it can provide a means to find data very quickly.

Introduction...

- SQL Server does not need to have indexes on small tables to retrieve data.
 - A small table can simply be scanned to find the piece of data that is requested efficiently.
 - In a **table scan** SQL Server reads all the pages.
- SQL Server organizes indexes in a very similar manner as an index at the back of a book or contents of page of a book.

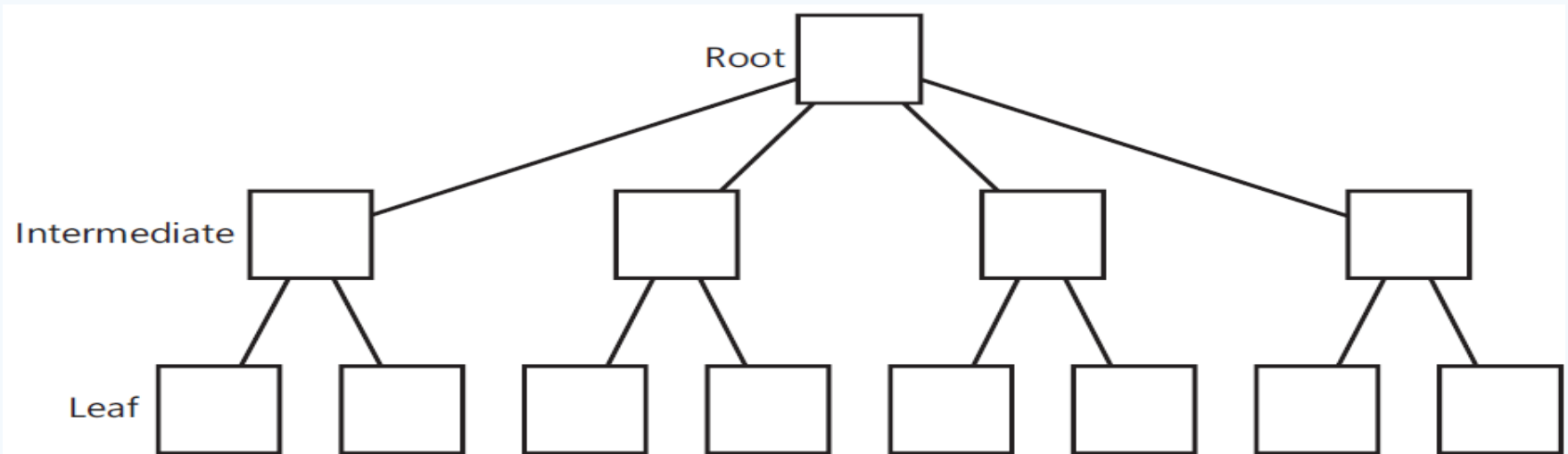
👉 Index structure

Index Structure

- ❖ SQL Server uses a **balanced tree (B-tree)** data structure to build and maintain indexes.
- ❖ B-tree is a symmetric, linear data structure, that enables searching very large volumes of data with a small number of read operations.
- ❖ An index can contain root, intermediate, and leaf pages.
- ❖ An index has a single root page defined at the top of the index structure.
- ❖ An index can have one or more levels of intermediate pages, but it is optional.

Index Structure...

- ❖ If all the entries do not fit on a single page, the index can create one or more pages at intermediate and leaf levels.
- An example of a B-tree structure is shown in figure below.



Index Levels

- ❖ The number of levels in an index and the number of pages within each level of an index are determined by simple mathematics.
- ❖ A data page in SQL Server is 8,192 bytes in size, which can be used to store up to 8,060 bytes of actual user data.
- ❖ Based on the number of bytes required to store an index key, determined by the data type, you can calculate the number of rows per page that are stored by using simple division.

Example

- If you build an index on an INT column, each row in the table will require 4 bytes of storage in the index.
- Number of int rows: $8060 \text{ Bytes} / 4 \text{ Bytes} = 2015$ rows.
- If the table contains only 1,200 rows of data, you need 4,800 bytes of storage.
- Because all the entries would fit on a single page of data, the index would have a single page that would be the root page as well as the leaf page.

Example...

- In fact, you could store 2,015 rows in the table and still allocate only a single page to the index.
- As soon as you add the 2,016th row, however, all the entries can no longer fit on a single page, so two additional pages are allocated to the index in a process called **page splitting**.

Designing Indexes

- It is frequently useful to create indexes on primary and foreign keys.
- Avoid indexing an attribute or relation that is frequently updated.
- Updating a table with indexes takes more time than updating a table without indexes (because the indexes also need an update).
- So you should only create indexes on columns that will be frequently searched against and sorted frequently.

Types of Indexes

- The different types of indexes are:
 - Clustered Indexes (primary relational indexes)
 - Nonclustered Indexes (secondary relational index)
 - XML Indexes
 - ✓ Created for XML data type.
 - Spatial Indexes
 - ✓ Created against a spatial column that is either geography or geometry data types.

Clustered Indexes

- Created by default on a primary key column.
- The column(s) defined for the clustered index are referred to as the **clustering key**.
- Enable rows to be stored in a logical order.
- Only one clustered index per table.
- Analogous to the contents of page of a book.
- All data are stored in the leaf level nodes.
- As SQL Server traverses the clustered index page chain, each row of data is read in clustering key order.

Clustered Indexes (Cont'd...)

- Provides excellent searching ability.
- Clustered index eliminates forwarding pointers.
- The simplified syntax to create clustered index is:

CREATE CLUSTERED INDEX indexName
ON tableName(columnNames);

- DROP INDEX syntax in SQL Server:

DROP INDEX tableName.indexName;

Nonclustered Indexes

- Created by default on secondary (unique) keys.
- Do not impose a sort order on the table, so you can create multiple nonclustered indexes per table.
- Separate objects to the table.
- Analogous to an index at the back of a book.
- The leaf level of a nonclustered index contains a pointer to the table where the rest of the columns can be found.
- If a clustered index exists on the table, the leaf level of the nonclustered index points at the clustering key.

Nonclustered Indexes (Cont'd...)

- If a clustered index does not exist on the table, the leaf level of the nonclustered index points at the row of data in the table.
- The simplified syntax to create (unique) nonclustered index is:

CREATE UNIQUE NONCLUSTERED INDEX

indexName **ON** tableName(columnNames);

- DROP INDEX syntax in SQL Server:

DROP INDEX tableName.indexName;

Questions

- 1) What will happen to the clustered index page if you apply the following operations to tables in your database?
 - a) Insert
 - b) Update
 - c) Delete
 - d) Select
- 2) Select queries related to the clustering key can **seek** or **scan**. What is the difference between index seek and index scan?

Index Management and Maintenance

- Over time, when data changes cause index pages to split, index fragmentation will occur.
 - **Internal fragmentation:** when pages are not full.
 - **External fragmentation:** occurs when pages are not in a logical sequence.
- Detecting fragmentation
 - Index properties in SQL Server Management Studio
- You can use the ALTER INDEX statement to remove the fragmentation (to defragment an index).

Index Management and Maintenance...

- When you defragment an index, you can use either the **REBUILD** or **REORGANIZE** options.
- The **REBUILD** option rebuilds all levels of the index.
- The **REORGANIZE** option removes fragmentation by sorting the leaf level pages.
- Syntax:

```
ALTER INDEX { index_name | ALL }  
ON tableName  
REBUILD | REORGANIZE;
```

Disabling and Enabling an Index

- Syntax to disable an index:

```
ALTER INDEX { index_name | ALL }  
ON tableName  
DISABLE;
```

- If a clustered index is disabled, the entire table becomes inaccessible.
- To enable an index, it must be rebuilt as follow:

```
ALTER INDEX { index_name | ALL }  
ON tableName  
REBUILD;
```